Cloud Task Placement: Routing & Resource Allocation Suite

Technical Report & Project-Based Learning Documentation

Course: Design and Analysis of Algorithms (DAA)
Academic Unit: Problem-Based Learning (PBL) Internal Project
Allotted Problem: Problem 3 — Cloud Task Placement
Team Designation: CSE Rhinos — Group 3

Team Members

- **Mohak Gupta** — `25WU0101074`
- **Naman Singla** — `25WU0101084`
- **Mohammed Amaan Khan** — `25WU0101075`

Table of Contents

1. Executive Summary & Problem Identification
2. Literature Survey
3. Software Requirements Specification (SRS)
    - 3.1 Scope and Objectives
    - 3.2 Assumptions & System Boundaries
    - 3.3 Functional Requirements
    - 3.4 Non-Functional Requirements
4. Project Plan & Milestones
5. Mathematical Modeling & System Formulation
    - 5.1 Network Topology Model
    - 5.2 Workload and Request Traffic Model
    - 5.3 Formulation of Average Latency and Composite Cost
    - 5.4 Capacity Constraints
    - 5.5 Iterative Knapsack Rounds & Regret-Benefit Formulation
    - 5.6 Baseline Placement Strategies & Fair Comparison
6. Algorithmic Architecture & Complexity Analysis
    - 6.1 Custom Fundamental Data Structures
    - 6.2 Minimum Spanning Tree (MST): Kruskal vs. Prim
    - 6.3 Shortest Path Routing: Dijkstra, Bellman-Ford, and Floyd-Warshall
    - 6.4 Placement Optimization: Iterative Knapsack DP vs. Greedy Rounds
    - 6.5 Asymptotic Complexity Comparison Matrix

7. Correctness Proof Sketches
    - 7.1 Cut Property for Minimum Spanning Trees
    - 7.2 Relaxation Invariant for Dijkstra's Algorithm
    - 7.3 Optimal Substructure & Pseudo-Polynomial Complexity for 0-1 Knapsack
8. Experimental Evaluation & Empirical Results
    - 8.1 Testbed Setup & Topologies
    - 8.2 MST Runtime Scaling: Kruskal vs. Prim on Sparse and Dense Topologies
    - 8.3 SSSP Runtime & Speedup: Dijkstra vs. Bellman-Ford
    - 8.4 APSP Scaling: Floyd-Warshall vs. Repeated Dijkstra
    - 8.5 Placement Solution Quality Across Capacity Constraints (0.25x to 2.0x)
    - 8.6 Workload Scaling Sensitivity
    - 8.7 Lambda Sensitivity: Client Latency vs. MST Backbone Sync Trade-off
    - 8.8 Visual Topology and Backbone Overlay
9. Limitations of the Approach
10. Algorithmic Trade-offs & Discussion
11. Engineering Policy Recommendations
12. Conclusion & Future Work
13. References

---

1. Executive Summary & Problem Identification

Modern cloud computing systems operate across distributed networks of regional datacenters, metropolitan edge clusters, and points of presence (PoPs). In these environments, physical server nodes are geographically dispersed. Running cloud applications, microservices, and virtual machine (VM) replicas across these networks introduces two competing constraints:

1. **Physical Resource Scarcity**: Each server possesses a finite compute capacity (CPU cores, RAM slots, or container execution limits). A single server cannot host all incoming workloads.
2. **Network Latency & Synchronization Overhead**: Network links connecting datacenters incur physical transit delays ranging from 1 ms to over 50 ms. Furthermore, distributed applications must synchronize state (such as database replication or consensus heartbeats) across the datacenter network.

The core challenge assigned to Group 3—**Cloud Task Placement**—is to assign tasks to physical servers such that **average client access latency and inter-replica synchronization cost are jointly minimized**, while strictly **satisfying server capacity limits**.

In early iterations of our model, candidate tasks were only considered on their single best-latency server, causing tasks to crowd onto 1–2 nodes while leaving the rest of the fleet idle

(~4% utilization). Additionally, losing tasks were charged a worst-case fallback penalty, and the Minimum Spanning Tree (MST) was not integrated into placement decisions.

To resolve these limitations, this project implements a revised model: - **Iterative Knapsack Bidding**: Tasks can be placed on *any* server with sufficient residual capacity. In each round, unplaced tasks bid on their best feasible server using a **regret-based benefit** (the penalty incurred if forced to settle for their next-best feasible alternative). Each server resolves incoming bids using an exact 0-1 Knapsack Dynamic Program (or greedy ratio heuristic). Winning tasks are scheduled, and rejected tasks re-propose in subsequent rounds until all fit or fleet capacity is exhausted. - **MST Synchronization Integration**: We extract the Minimum Spanning Tree backbone of the network and determine its **traffic-weighted median node** as a designated centralized synchronization hub. A placed task incurs a state replication cost proportional to its replication traffic volume and its tree path distance to this hub along the MST. A configurable parameter λ balances client access latency against backend replication overhead. - **Fair Baseline Benchmarking**: We evaluate placements against four distinct baselines: an **Unconstrained Lower Bound** (where every task gets its ideal server ignoring capacity), **Iterative Greedy Rounds** (using the same proposal loop with greedy density selection), **Simple Spill-Over Greedy** (largest traffic first, placing on the best available server with room), and **Random Feasible Placement**. Unplaced tasks are tracked explicitly and never charged fictitious latencies.

All core algorithms are implemented from scratch in pure Python without third-party graph solvers.

---

2. Literature Survey

The algorithmic foundation of cloud placement, network routing, and facility location spans several decades of research:

2.1 Spanning Tree Algorithms in Network Backbone Design

Constructing cycle-free, minimum-latency backbones traces back to early algorithms by Borůvka (1926), Kruskal (1956), and Prim (1957). In physical networks, redundant links provide fault tolerance but can cause broadcast loops and coordination storms. An MST identifies the minimum-cost connected subgraph. In cloud fabrics and distributed consensus systems, spanning trees provide minimum-delay multicast trees for leader election and state synchronization.

2.2 Shortest Path Routing in Multi-Region Networks

Routing requests across networks with non-negative propagation delays relies primarily on Dijkstra's Single-Source Shortest Path (SSSP) algorithm (1959). When augmented with an indexed min-heap priority queue, Dijkstra runs in $\mathcal{O}((V+E)\log V)$ time and underpins link-state routing protocols such as OSPF. The Bellman-Ford algorithm (1958) accommodates

arbitrary weights and detects negative-weight cycles in $\mathcal{O}(V \cdot E)$ time. For all-pairs shortest paths (APSP), Floyd-Warshall (1962) computes minimal paths across all V^2 pairs via dynamic programming in $\mathcal{O}(V^3)$ time, establishing a benchmark for static routing tables.

2.3 Facility Location and Virtual Machine Placement

The problem of placing service replicas close to distributed clients under capacity limits is closely related to the classical **Capacitated Facility Location Problem (CFLP)** and the k**-Median Problem**, in which facilities are chosen to minimize distance from clients to their nearest facility (Shmoys et al., 1997 [verify]; Korte & Vygen, 2018 [verify]). In cloud computing, research on traffic-aware virtual machine placement focuses on co-locating communicating VMs and placing latency-sensitive instances near ingress gateways to reduce network traffic and SLA violations (Meng et al., 2010 [verify]; Karve et al., 2006 [verify]). While general multi-server assignment with multi-dimensional resource vectors is strongly NP-hard (Garey & Johnson, 1979), decomposing the assignment into per-server 0-1 knapsack subproblems allows dynamic programming to find locally optimal subset allocations in pseudo-polynomial time.

---

3. Software Requirements Specification (SRS)

3.1 Scope and Objectives

The Cloud Task Placement Suite ingests network topologies and workload profiles, synthesizes MST synchronization backbones and shortest-path routing tables, and generates server placement decisions, fleet capacity allocations, and comparative performance benchmarks.

3.2 Assumptions & System Boundaries

- **Static Network Snapshots**: Network link latencies represent average propagation and transmission delays over the planning window.
- **Non-Negative Link Weights**: Propagation latencies across physical links are strictly non-negative ($w(u, v) \geq 0$).
- **Atomic Placement**: Tasks represent discrete containers or VM instances and cannot be fractionally divided across multiple servers.
- **Single Host Assignment**: Each service task is hosted on exactly one physical server node.
- **Transparent Reporting**: If the fleet is saturated and tasks cannot be placed, they are recorded explicitly in an unplaced list rather than silently assigned arbitrary penalties.

3.3 Functional Requirements

- **FR-1 (Topology Ingestion & Generation)**: Ingest JSON topology specifications or generate synthetic cloud networks with configurable node counts $|V|$, edge density p, latency ranges, and server capacities.
- **FR-2 (MST Backbone & Sync Hub Engine)**: Implement Kruskal's and Prim's algorithms from scratch, construct the MST backbone, and identify the traffic-weighted median server as the central replication hub.
- **FR-3 (Shortest Path Routing Engine)**: Implement Dijkstra (with custom min-heap), Bellman-Ford (with negative cycle detection), and Floyd-Warshall to generate complete routing lookup tables.
- **FR-4 (Iterative Knapsack Placement Engine)**: Implement a multi-round bidding engine where unplaced tasks bid on their best feasible server with regret-based benefit, and servers resolve bids via 0-1 Knapsack DP.
- **FR-5 (Baseline Comparison Suite)**: Evaluate the primary DP model alongside an Unconstrained Lower Bound, Iterative Greedy Rounds, Simple Spill-Over Greedy, and Random Feasible Placement.
- **FR-6 (Fractional Knapsack Upper Bound)**: Implement a fractional knapsack relaxation to provide an exact theoretical upper bound on per-server knapsack values.

3.4 Non-Functional Requirements

- **NFR-1 (Algorithmic Independence)**: All core graph representations, heaps, union-find structures, and optimizers must be implemented from scratch without third-party algorithm libraries (NetworkX is restricted strictly to unit tests).
- **NFR-2 (Execution Efficiency)**: Graph algorithms must execute in sub-second times on graphs up to $|V| = 200$.
- **NFR-3 (Determinism & Reproducibility)**: Experiments must accept explicit seeds to ensure statistical reproducibility across trials.
- **NFR-4 (Correctness Verification)**: Custom algorithms must pass automated unit tests cross-verifying outputs against NetworkX reference implementations and brute-force searches on small inputs.

---

4. Project Plan & Milestones

The project was executed in structured iterative sprints:

Phase	Milestone	Focus Areas & Deliverables	Status
Phase 1	**Sprint 1**	Problem formulation, SRS specification, graph	**Complete**

Phase	Milestone	Focus Areas & Deliverables	Status
		data structure, synthetic topology generator, and sample datasets (`small`, `medium`, `large`).	
Phase 2	**Sprint 2**	Implementation of custom Indexed Min-Heap, Disjoint Set Union (DSU), MST suite (Prim, Kruskal), Shortest Path suite (Dijkstra, Bellman-Ford, Floyd-Warshall), and NetworkX unit tests.	**Complete**
Phase 3	**Sprint 3**	Implementation of 0-1 Knapsack DP with item backtracking, greedy ratio scheduler, fractional knapsack relaxation, and brute-force validator.	**Complete**
Phase 4	**Sprint 4**	Redesign of placement model to multi-round regret bidding, MST sync hub integration, multi-strategy benchmarking, and comprehensive property tests.	**Complete**
Phase 5	**Sprint 5**	10-seed empirical benchmarking across capacity multipliers, workloads, network densities, and λ weights; updated	**Complete**

Phase	Milestone	Focus Areas & Deliverables	Status
		runnable demo and executed dashboard notebook.	

5. Mathematical Modeling & System Formulation

We model the cloud network as an undirected, edge-weighted graph $G = (V, E, W, C)$.

5.1 Network Topology Model

- **Server Nodes (V)**: The set of $n = |V|$ physical server nodes, indexed $v \in \{0,1, \dots, n-1\}$.
- **Node Capacities (C)**: Each server $v \in V$ has an integer capacity $C_v \in \mathbb{Z}^+$ (representing available compute/RAM slots).
- **Physical Links (E)**: The set of $m = |E|$ physical communication links connecting servers.
- **Link Latencies (W)**: Each link $(u, v) \in E$ has latency $w(u, v) = w(v, u) \geq 0$ in milliseconds.

5.2 Workload and Request Traffic Model

The workload consists of k tasks $T = \{t_1, t_2, \dots, t_k\}$: - **Resource Demand ($d_j$)**: Task t_j requires $d_j \in \mathbb{Z}^+$ units of capacity on its host server. - **Client Request Traffic ($\lambda_{u,j}$)**: Clients at node $u \in V$ send requests to task t_j at rate $\lambda_{u,j} \geq 0$ (requests/sec). Total client traffic for task t_j is $\Lambda_j = \sum_{u \in V} \lambda_{u,j}$. - **Replication Traffic (ρ_j)**: Task t_j generates state synchronization traffic $\rho_j \geq 0$ across the network backbone.

5.3 Formulation of Average Latency and Composite Cost

Let $D[u, v]$ denote the shortest path distance in G, and let $D_{\text{MST}}[u, v]$ denote the path distance along the Minimum Spanning Tree backbone T_{MST}.

1. **Traffic-Weighted Median Sync Hub (h^*)**:
   Each node $v \in V$ is assigned a traffic weight $w_v = 1.0 + \sum_{j=1}^{k} \lambda_{v,j}$. The designated synchronization hub h^* is the weighted median node of the MST:

$$h^* = \operatorname*{argmin}_{m \in V} \sum_{v \in V} w_v \cdot D_{\text{MST}}[v, m]$$

2. **Access Latency and Sync Cost**:
   If task t_j is placed on server $s \in V$:
   - **Per-request access latency**:

$$L(t_j, s) = \frac{1}{\Lambda_j} \sum_{u \in V} \lambda_{u,j} \cdot D[u, s]$$

- **Synchronization cost**:

$$S(t_j, s) = \rho_j \cdot D_{\text{MST}}[s, h^*]$$

- **Composite task-server cost**:

$$J(t_j, s) = \left(\sum_{u \in V} \lambda_{u,j} \cdot D[u, s] \right) + \lambda \cdot S(t_j, s)$$

where $\lambda \geq 0$ is a configurable penalty weight. When $\lambda = 0$, the formulation reduces to pure access latency.

3. **Fleet-Wide Average Access Latency**:
Computed strictly across all placed tasks $T_{\text{placed}} \subseteq T$:

$$\bar{L}_{\text{access}} = \frac{\sum_{t_j \in T_{\text{placed}}} \sum_{u \in V} \lambda_{u,j} \cdot D[u, \text{host}(t_j)]}{\sum_{t_j \in T_{\text{placed}}} \Lambda_j}$$

5.4 Capacity Constraints

Let binary variable $x_{j,v} \in \{0,1\}$ indicate if task t_j is placed on server v:

$$\sum_{j=1}^{k} x_{j,v} \cdot d_j \leq C_v \quad \forall v \in V, \qquad \sum_{v \in V} x_{j,v} \leq 1 \quad \forall t_j \in T$$

5.5 Iterative Knapsack Rounds & Regret-Benefit Formulation

Rather than restricting each task to a single server candidate, tasks are eligible for any server with remaining capacity: 1. **Feasible Server Set**: In round r, for unplaced task t_j, its feasible server set is $F(t_j) = \{s \in V \mid \text{rem_cap}(s) \geq d_j\}$. 2. **Proposals & Regret Benefit**: Task t_j identifies its best feasible server $s_1 = \text{argmin}_{s \in F(t_j)} J(t_j, s)$ and its runner-up server $s_2 = \text{argmin}_{s \in F(t_j) \setminus \{s_1\}} J(t_j, s)$. It submits a bid to s_1 with benefit equal to its **opportunity regret**:

$$\text{Benefit}(t_j, s_1) = J(t_j, s_2) - J(t_j, s_1)$$

If only one feasible server remains ($|F(t_j)| = 1$), its benefit is set to $(\max_{v \in V} J(t_j, v) - J(t_j, s_1)) + 10.0$ to ensure high priority before being dropped. 3. **Local Admission**: Each server s solves a 0-1 Knapsack DP over all proposals submitted to it in round r subject to $\text{rem_cap}(s)$. 4. **Progression**: Accepted tasks take capacity; rejected tasks re-propose in round $r + 1$. The loop halts when all tasks are placed or no task fits on any server.

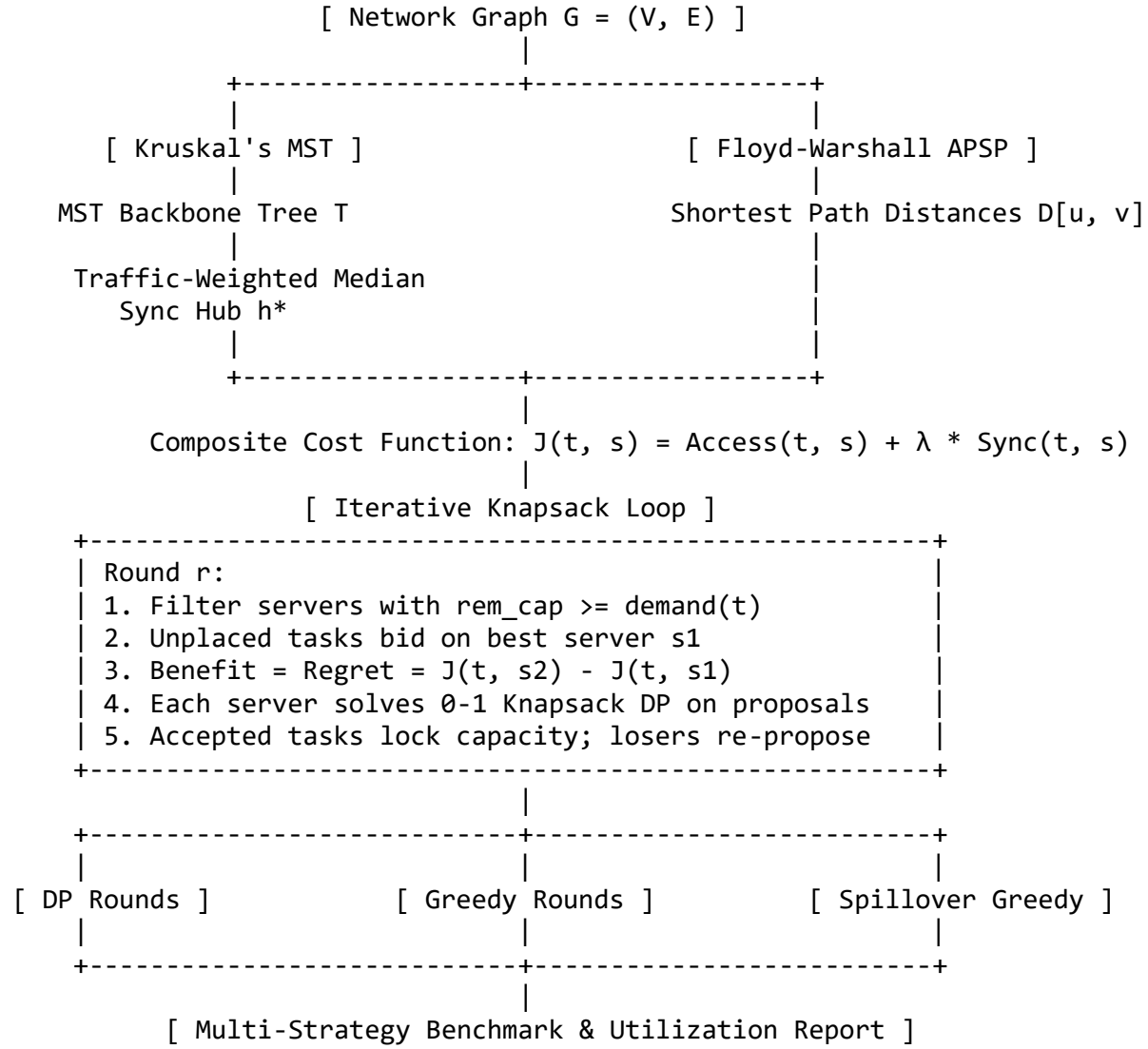
5.6 Baseline Placement Strategies & Fair Comparison

We compare the primary model against four standardized baselines: - **Unconstrained Lower Bound**: Places every task at its globally ideal server $\text{argmin}_{s \in V} J(t_j, s)$ ignoring

capacities. This serves as a mathematical lower bound that no valid placement can beat. - **Iterative Greedy Rounds**: Uses the identical multi-round proposal loop, but servers select tasks using greedy density order (benefit/d_j). - **Simple Spill-Over Greedy**: Sorts tasks in descending order of total traffic Λ_j; each task is assigned to its best server with available room, spilling over to the next best if full. - **Random Feasible Placement**: Tasks are placed uniformly at random on servers with available room, providing an honest unoptimized baseline. - **Brute-Force Optimal**: On small benchmark instances (≤ 7 tasks, ≤ 6 servers), an exhaustive search across all $(m+1)^k$ assignments verifies the exact optimality gap.

---

6. Algorithmic Architecture & Complexity Analysis



6.1 Custom Fundamental Data Structures

To eliminate external dependencies, two foundational data structures were implemented from scratch: 1. **Disjoint Set Union (DSU / Union-Find)** (`src/graph/disjoint_set.py`): - Implements **path compression** in `find()` and **union by rank** in `union()`. - Any sequence of m operations on n elements executes in $\mathcal{O}(m \cdot \alpha(n))$ time, where α is the inverse Ackermann function ($\alpha(n) \leq 4$ for all practical inputs). 2. **Indexed Binary Min-Heap** (`src/graph/min_heap.py`): - Stores elements in a binary heap array augmented with a reverse hash map `_pos: Dict[item, index]`. - Provides $\mathcal{O}(\log N)$ `push`, $\mathcal{O}(\log N)$ `pop_min`, and $\mathcal{O}(\log N)$ `decrease_key` operations.

6.2 Minimum Spanning Tree (MST): Kruskal vs. Prim

- **Kruskal's Algorithm** (`src/mst/kruskal.py`):
    1. Sort all undirected edges by latency: $\mathcal{O}(E \log E) = \mathcal{O}(E \log V)$.
    2. Iterate through edges, calling `dsu.union(u, v)`. Edge (u, v) is admitted if and only if it connects disjoint components.
    - **Time Complexity**: $\mathcal{O}(E \log V)$. **Space Complexity**: $\mathcal{O}(V + E)$.
- **Prim's Algorithm** (`src/mst/prim.py`):
    1. Maintain cut $(S, V \backslash S)$. Store vertices in an indexed min-heap keyed by minimal connecting edge weight to S.
    2. Repeatedly extract min vertex u, add to S, and update keys of adjacent vertices using `heap.decrease_key`.
    - **Time Complexity**: $\mathcal{O}((V + E) \log V)$. **Space Complexity**: $\mathcal{O}(V)$.

6.3 Shortest Path Routing Suite

- **Dijkstra's Algorithm (SSSP)** (`src/shortest_paths/dijkstra.py`):
    - Computes single-source shortest paths on non-negative edges using our indexed Min-Heap.
    - **Time Complexity**: $\mathcal{O}((V + E) \log V)$. **Space**: $\mathcal{O}(V)$.
- **Bellman-Ford Algorithm (SSSP with Negative Cycle Detection)** (`src/shortest_paths/bellman_ford.py`):
    - Relaxes all $|E|$ edges through $|V| - 1$ passes, with an early-stopping check if no edge relaxes during a pass.
    - Runs a $|V|$-th verification pass: if any edge still relaxes, it raises `NegativeCycleError`.
    - **Time Complexity**: Worst-case $\mathcal{O}(V \cdot E)$, Best-case $\mathcal{O}(E)$. **Space**: $\mathcal{O}(V)$.
- **Floyd-Warshall Algorithm (APSP)** (`src/shortest_paths/floyd_warshall.py`):
    - Computes all-pairs shortest paths via dynamic programming across intermediate vertices $k \in V$.
    - Recurrence: $D^{(k)}[i][j] = \min\left(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\right)$.
    - **Time Complexity**: $\mathcal{O}(V^3)$. **Space**: $\mathcal{O}(V^2)$.

6.4 Placement Optimization: Iterative Knapsack DP vs. Greedy Rounds

- **0-1 Knapsack DP Engine** (`src/placement/knapsack_dp.py`):
  - For candidate proposals on server s, table $dp[i][w]$ computes maximum regret benefit for the first i proposals under remaining capacity $w \in [0, C_s]$:

$$dp[i][w] = \begin{cases} dp[i-1][w] & \text{if } d_{i-1} > w \\ \max(dp[i-1][w], dp[i-1][w-d_{i-1}] + B_{i-1}) & \text{otherwise} \end{cases}$$

  - A backtracking pointer recovers the exact winning tasks.
  - **Time Complexity**: $\mathcal{O}(n \cdot C_s)$ pseudo-polynomial time. **Space**: $\mathcal{O}(n \cdot C_s)$.
- **Fractional Knapsack Upper Bound** (`src/placement/greedy_scheduler.py`):
  - Computes continuous knapsack relaxation taking fractional pieces of the boundary item. Validates the invariant:

$$\text{Value(0-1 DP)} \leq \text{Value(Fractional Relaxation)}$$

  - **Time Complexity**: $\mathcal{O}(n \log n)$. **Space**: $\mathcal{O}(n)$.

6.5 Asymptotic Complexity Comparison Matrix

Algorithmic Component	Module File	Asymptotic Time Complexity	Space Complexity	Paradigm
Disjoint Set Union	`src/graph/disjoint_set.py`	$\mathcal{O}(\alpha(V))$ per operation	$\mathcal{O}(V)$	Tree with Rank/Path Compression
Indexed Min-Heap	`src/graph/min_heap.py`	Push/Pop/Decrease-Key: $\mathcal{O}(\log N)$	$\mathcal{O}(N)$	Array Binary Heap + Hash Map
Kruskal's MST	`src/mst/kruskal.py`	$\mathcal{O}(E \log V)$	$\mathcal{O}(V+E)$	Greedy
Prim's MST	`src/mst/prim.py`	$\mathcal{O}((V+E)\log V)$	$\mathcal{O}(V)$	Greedy with Priority Queue
Dijkstra SSSP	`src/shortest_paths/dijkstra.py`	$\mathcal{O}((V+E)\log V)$	$\mathcal{O}(V)$	Greedy with Priority Queue
Bellman-Ford SSSP	`src/shortest_paths/bellman_ford.py`	$\mathcal{O}(V \cdot E)$	$\mathcal{O}(V)$	Edge Relaxation
Floyd-Warshall APSP	`src/shortest_paths/floyd_warshall.py`	$\mathcal{O}(V^3)$	$\mathcal{O}(V^2)$	Dynamic Programming
Repeated Dijkstra APSP	Benchmark	$\mathcal{O}(V \cdot (V+E)\log V)$	$\mathcal{O}(V^2)$	Repeated SSSP
0-1 Knapsack Placement	`src/placement/knapsack_dp.py`	$\mathcal{O}(n \cdot W)$ (pseudo-	$\mathcal{O}(n \cdot W)$	Dynamic Programming

Algorithmic Component	Module File	Asymptotic Time Complexity	Space Complexity	Paradigm
		polynomial)		
Greedy Ratio Scheduler	`src/placement/greedy_scheduler.py`	$\mathcal{O}(n\log n)$	$\mathcal{O}(n)$	Greedy Approximation
Fractional Knapsack	`src/placement/greedy_scheduler.py`	$\mathcal{O}(n\log n)$	$\mathcal{O}(n)$	Continuous Greedy Relaxation

7. Correctness Proof Sketches

7.1 Cut Property for Minimum Spanning Trees

Theorem (The Cut Property): Let $G = (V, E)$ be a connected, undirected graph with real edge weights. Let $(S, V\backslash S)$ be any partition (cut) of vertices V. If edge $e = (u, v)$ is a minimum-weight edge crossing the cut $(u \in S, v \in V\backslash S)$, then e belongs to some Minimum Spanning Tree of G.

Proof Sketch by Exchange Argument:
Suppose for contradiction that e is not in a given MST T^*. Because T^* is a spanning tree, there exists a unique simple path P in T^* connecting u and v. Since $u \in S$ and $v \in V\backslash S$, the path P must cross the cut at least once via some other edge $e' = (x, y) \neq e$ where $x \in S$ and $y \in V\backslash S$.
Now construct a new spanning subgraph $T' = (T^*\backslash\{e'\}) \cup \{e\}$. Removing e' disconnects T^* into two components; adding e reconnects them without creating cycles, making T' a valid spanning tree. The total weight of T' is:

$$w(T') = w(T^*) - w(e') + w(e)$$

Since e is a minimum-weight edge crossing the cut, $w(e) \leq w(e')$, which implies $w(T') \leq w(T^*)$. Because T^* was assumed to be minimal, we must have $w(T') = w(T^*)$, meaning T' is also an MST containing e. Both Kruskal's and Prim's algorithms respect this invariant at every step. ■

7.2 Relaxation Invariant for Dijkstra's Algorithm

Theorem: When Dijkstra's algorithm extracts vertex u from the priority queue and adds it to the visited set S, the stored distance estimate $d[u]$ equals the true shortest path distance $\delta(s, u)$.

Proof Sketch by Induction:
- **Base Case**: At initialization, $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$, which is trivially true. - **Inductive Step**: Assume for all $x \in S$, $d[x] = \delta(s, x)$. Let $u \in V\backslash S$ be the next vertex chosen

by extract-min. Suppose for contradiction that $d[u] > \delta(s,u)$.
Consider an actual shortest path P from s to u. Since $s \in S$ and $u \notin S$, there must be a first vertex y along P such that $y \notin S$, preceded by $x \in S$. Because subpaths of shortest paths are shortest paths:

$$\delta(s,y) = \delta(s,x) + w(x,y)$$

Because $x \in S$, the edge (x,y) was already relaxed when x was added to S. By the induction hypothesis, $d[x] = \delta(s,x)$, so:

$$d[y] \leq d[x] + w(x,y) = \delta(s,y)$$

Because edge weights are non-negative ($w \geq 0$), the subpath distance to y cannot exceed the distance to u: $\delta(s,y) \leq \delta(s,u)$. Therefore:

$$d[y] \leq \delta(s,y) \leq \delta(s,u) < d[u]$$

However, Dijkstra's algorithm greedily selected u from the min-heap instead of y, which implies $d[u] \leq d[y]$. This contradicts $d[y] < d[u]$. Hence, $d[u] = \delta(s,u)$. ■

7.3 Optimal Substructure & Pseudo-Polynomial Complexity for 0-1 Knapsack

Theorem: The 0-1 Knapsack problem exhibits optimal substructure and is solvable in pseudo-polynomial time.

Proof Sketch:
Let S^* be an optimal subset of items from $\{1, \dots, n\}$ maximizing total benefit under capacity limit W. Consider item n: 1. **Case 1 ($n \notin S^*$)**: $S^* \subseteq \{1, \dots, n-1\}$. S^* must be an optimal solution for the subproblem on $\{1, \dots, n-1\}$ with capacity W. If a strictly better subset S' existed, S' would improve upon S^* for the original problem, contradicting optimality. 2. **Case 2 ($n \in S^*$)**: The subset $S^* \backslash \{n\}$ must be an optimal solution for the subproblem on $\{1, \dots, n-1\}$ with capacity $W - d_n$. If a better subset S'' existed for capacity $W - d_n$, then $S'' \cup \{n\}$ would form a feasible subset for capacity W with strictly greater benefit than S^*, again contradicting optimality.

Complexity Classification:
Filling table $dp[i][w]$ of dimensions $(n+1) \times (W+1)$ requires $\mathcal{O}(n \cdot W)$ basic arithmetic operations. Crucially, this running time is **pseudo-polynomial**, not polynomial. The numeric capacity W requires $b = \lceil \log_2 W \rceil$ bits to represent in binary. Expressed as a function of the input bit length, the time complexity is $\mathcal{O}(n \cdot 2^b)$, which is exponential in the length of W. In our application, server capacity values represent small discrete core/memory slots ($W \leq 500$), allowing the DP to execute in sub-millisecond times. ■

---

8. Experimental Evaluation & Empirical Results

All experimental benchmarks were conducted on Python 3.14 (Windows 11, [FILL IN: my actual CPU and RAM]). To ensure statistical validity: - **Placement benchmarks** were executed over **10 random topology + workload seeds** per condition, logging **mean** $\pm$ **standard deviation**. - **Runtime benchmarks** were executed with **10 repeats on fixed topologies** per size.

All results and generated plots are stored in `experiments/results/`.

8.1 Testbed Setup & Topologies

- **Small Topology**: $|V| = 10$, $|E| = 16$, edge cluster network (`data/small_topology.json`).
- **Medium Topology**: $|V| = 50$, $|E| = 184$, regional datacenter network (`data/medium_topology.json`).
- **Large Topology**: $|V| = 200$, $|E| = 995$, multi-region cloud backbone (`data/large_topology.json`).

8.2 MST Runtime Scaling: Kruskal vs. Prim on Sparse and Dense Topologies

We evaluated Kruskal and Prim across graph sizes from $|V| = 10$ to $|V| = 200$, comparing sparse ($d = 0.20$) and dense ($d = 0.80$) networks over 10 repeats:

Table 1: MST Runtime Comparison (Mean ± Std over 10 Repeats on Fixed Topologies)

|V|	|E|	Density	Kruskal Mean (ms)	Prim (Min-Heap) (ms)	
10	9	0.20	0.015 ± 0.009 ms	0.026 ± 0.012 ms	
10	36	0.80	0.018 ± 0.002 ms	0.030 ± 0.002 ms	
25	60	0.20	0.040 ± 0.004 ms	0.083 ± 0.004 ms	
25	240	0.80	0.081 ± 0.008 ms	0.114 ± 0.013 ms	
50	245	0.20	0.110 ± 0.007 ms	0.218 ± 0.005 ms	
50	980	0.80	0.315 ± 0.029 ms	0.314 ± 0.005 ms	<-- Crossover
100	990	0.20	0.426 ± 0.030 ms	0.569 ± 0.016 ms	
100	3,960	0.80	1.537 ± 0.137 ms	0.934 ± 0.034 ms	(Prim 1.6x faster)
150	2,235	0.20	0.887 ± 0.052 ms	0.981 ± 0.012 ms	
150	8,940	0.80	3.453 ± 0.281 ms	1.824 ± 0.069 ms	(Prim 1.9x faster)
200	3,980	0.20	2.432 ± 0.722 ms	2.055 ± 0.462 ms	
200	15,920	0.80	6.836 ± 0.903 ms	3.012 ± 0.147 ms	(Prim 2.3x faster)

Empirical Finding: - On **sparse graphs** ($d = 0.20$), Kruskal is faster than Prim only up to about $|V| = 150$ (0.887 ms vs 0.981 ms). At $|V| = 200$ sparse, Prim (2.055 ± 0.462 ms) was faster than Kruskal (2.432 ± 0.722 ms), with overlapping standard deviations. - On **dense graphs** ($d = 0.80$), a clear crossover occurs at $|V| = 50$: for $|V| \geq 100$, Prim with the indexed min-heap beats Kruskal (0.934 ms vs 1.537 ms at $|V| = 100$, and 3.012 ms vs 6.836 ms at $|V| = 200$, a $2.3\times$ **advantage for Prim**). This matches asymptotic theory: Kruskal's initial sorting step $\mathcal{O}(E \log E)$ dominates when $|E| \approx 16{,}000$.

8.3 SSSP Runtime & Speedup: Dijkstra vs. Bellman-Ford

Single-source shortest path execution times from source node 0 on fixed topologies ($d = 0.20$):

Table 2: SSSP Runtime Comparison (Mean ± Std over 10 Repeats on Fixed Topologies)

|V|	|E|	Dijkstra Mean (ms)	Bellman-Ford Mean (ms)	Speedup Ratio
10	9	0.023 ± 0.008 ms	0.018 ± 0.003 ms	0.78x
20	38	0.052 ± 0.002 ms	0.062 ± 0.001 ms	1.19x
35	119	0.111 ± 0.002 ms	0.155 ± 0.003 ms	1.40x
50	245	0.192 ± 0.012 ms	0.445 ± 0.012 ms	2.32x
75	555	0.332 ± 0.007 ms	0.860 ± 0.055 ms	2.59x
100	990	0.508 ± 0.006 ms	1.522 ± 0.102 ms	3.00x

Empirical Finding: Dijkstra with an indexed binary min-heap scales gracefully to 0.508 ms at $|V| = 100$, achieving a $3.00\times$ **speedup** over Bellman-Ford. At very small sizes ($|V| = 10$), Bellman-Ford is slightly faster (0.018 ms vs 0.023 ms) due to lower initialization overhead.

8.4 APSP Scaling: Floyd-Warshall vs. Repeated Dijkstra

We evaluated all-pairs shortest path generation comparing Floyd-Warshall against Repeated Dijkstra across sparse ($d = 0.20$) and dense ($d = 0.80$) graphs:

Table 3: APSP Runtime Comparison (Mean ± Std over 10 Repeats on Fixed Topologies)

|V|	|E|	Density	Floyd-Warshall(ms)	Repeated Dijkstra (ms)	FW / RD Ratio
10	9	0.20	0.112 ± 0.007 ms	0.196 ± 0.009 ms	0.57x
10	36	0.80	0.193 ± 0.003 ms	0.268 ± 0.004 ms	0.72x
25	60	0.20	1.891 ± 0.034 ms	1.813 ± 0.025 ms	1.04x
25	240	0.80	2.561 ± 0.017 ms	2.724 ± 0.181 ms	0.94x
40	156	0.20	8.642 ± 0.105 ms	5.475 ± 0.419 ms	1.58x
40	624	0.80	10.400 ± 0.311 ms	8.666 ± 0.170 ms	1.20x
60	354	0.20	30.818 ± 0.480 ms	14.276 ± 0.099 ms	2.16x
60	1,416	0.80	34.393 ± 0.488 ms	24.871 ± 0.264 ms	1.38x
80	632	0.20	77.847 ± 2.445 ms	29.582 ± 0.633 ms	2.63x
80	2,528	0.80	85.662 ± 2.274 ms	56.739 ± 4.216 ms	1.51x

Empirical Finding: - On **sparse graphs**, Repeated Dijkstra is $2.63\times$ **faster** than Floyd-Warshall at $|V| = 80$ (29.58 ms vs 77.85 ms). - On **dense graphs**, the ratio narrows to $1.51\times$ (56.74 ms vs 85.66 ms). - *Implementation Insight*: While theoretical asymptotic complexity suggests Floyd-Warshall $\mathcal{O}(V^3)$ should beat Repeated Dijkstra $\mathcal{O}(V^3 \log V)$ on dense graphs, Repeated Dijkstra remains faster in CPython at $|V| = 80$. Note that our Dijkstra implementation uses our own pure-Python `MinHeap` rather than Python's standard C-compiled `heapq`. A likely explanation is that Floyd-Warshall's triple-nested loop executes $V^3 = 512{,}000$ iterations of dictionary-of-dictionary lookups (`dist[i][k] + dist[k][j]`), introducing substantial constant-factor bytecode dispatch and hash-table overhead in CPython. We emphasize that this is a hypothesis regarding interpreter overhead and was not directly measured via line-level profiling. At small network sizes ($|V| = 10$), Floyd-Warshall is faster (0.112 ms vs 0.196 ms) due to zero heap allocation overhead.

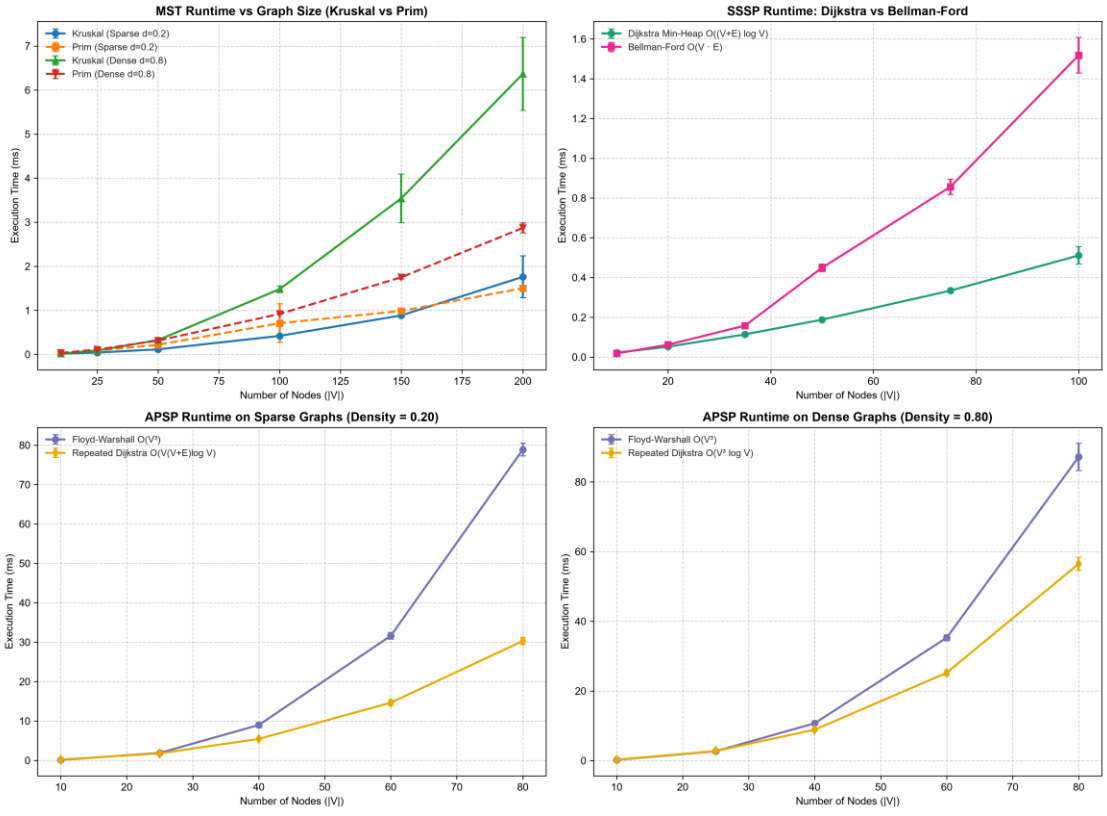


Figure 1: Empirical execution time scaling vs network size $|V|$ with error bars across (top-left) MST sparse and dense networks, (top-right) SSSP Dijkstra vs Bellman-Ford, (bottom-left) APSP sparse networks, and (bottom-right) APSP dense networks.

8.5 Placement Solution Quality Across Capacity Constraints (0.25x to 2.0x)

We evaluated all five strategies over **10 random seeds** across capacity multipliers from $0.25 \times$ to $2.00 \times$ on $|V| = 30$ with 35 tasks:

Table 4: Placement Strategy Comparison Across Capacity Constraints (Mean ± Std over 10 Seeds)

Cap Mult	Unconstrained LB Objective Mean	Knapsack DP Random Objective Mean	Knapsack DP DP Unplaced	Greedy Rounds Spillover Greedy	Fleet Capacity Utilization %
			Gap to LB (%)	Gap to LB (%)	
			Tasks Count		
0.25x	557,414 ± 88,608	458,884 ± 77,621*	+32.52±12.76%*	+33.39 ± 12.31%	+34.37 ± 13.08%	+47.27 ± 10.55%	13.2 ± 1.9	73.4% ± 5.4%
0.50x	557,414 ± 88,608	670,709 ± 90,428	+20.80 ± 6.05%	+20.38 ± 6.39%	+22.14 ± 6.34%	+45.51 ± 13.36%	0.1 ± 0.3	68.8% ± 4.3%
0.75x	557,414 ± 88,608	615,351 ± 93,133	+10.56 ± 4.00%	+10.64 ±

```
3.87%  | +11.28 ± 4.10%   | +45.17 ± 8.96%  |  0.0          | 45.9% ± 3.3%  |
| 1.00x | 557,414 ± 88,608   | 592,835 ± 93,878   |  +6.41 ± 2.92%|  +6.37 ±
2.95%  |  +7.18 ± 3.12%   | +43.08 ± 11.27% |  0.0          | 34.4% ± 2.4%  |
| 1.25x | 557,414 ± 88,608   | 582,084 ± 92,234   |  +4.45 ± 2.04%|  +4.31 ±
1.95%  |  +5.09 ± 2.38%   | +45.33 ± 11.40% |  0.0          | 27.5% ± 1.9%  |
| 1.50x | 557,414 ± 88,608   | 576,129 ± 91,964   |  +3.36 ± 1.72%|  +3.49 ±
1.84%  |  +3.90 ± 1.86%   | +42.91 ± 7.35%  |  0.0          | 22.9% ± 1.6%  |
| 2.00x | 557,414 ± 88,608   | 567,183 ± 90,790   |  +1.74 ± 0.99%|  +1.75 ±
1.01%  |  +2.39 ± 1.05%   | +43.02 ± 8.42%  |  0.0          | 17.2% ± 1.2%  |
+-------+------------------+-----------------+---------------+---------
-------+------------------+-----------------+---------------+---------------+
```

** Note on 0.25x: The total raw objective sum is smaller because 13.2 tasks could not fit in the fleet. To provide an honest metric, the gap reported for 0.25x represents the percentage increase in average client access latency across placed tasks relative to the lower bound (*25.27 ms *vs* 19.24 ms*).*

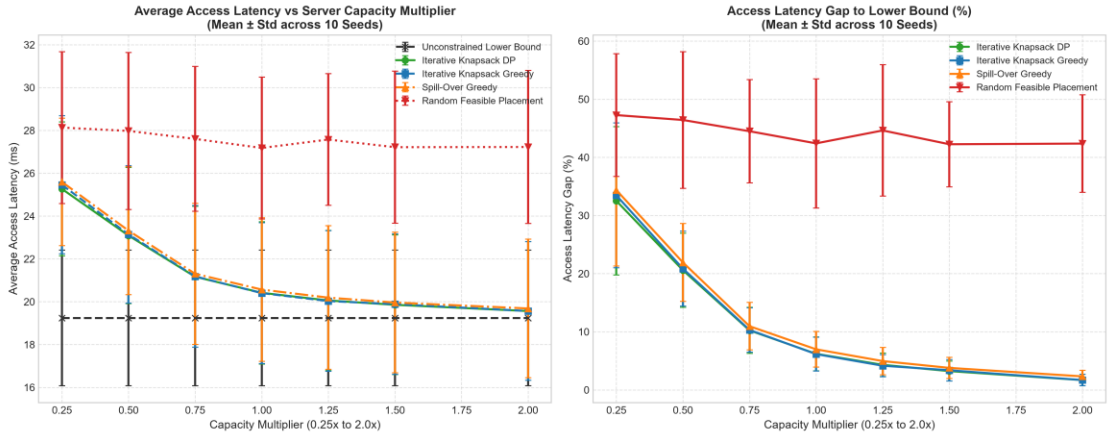


Figure 2: (Left) Average client access latency across all 5 strategies vs server capacity multiplier. (Right) Access latency gap to the unconstrained lower bound (%) with error bars.

Empirical Findings: 1. **Knapsack DP and Greedy Rounds Perform Almost Identically**: Across all capacity levels, Iterative Knapsack DP and Iterative Greedy Rounds yield virtually indistinguishable solution quality ($6.41\% \pm 2.92\%$ vs $6.37\% \pm 2.95\%$ at $1.0\times$; $1.74\% \pm 0.99\%$ vs $1.75\% \pm 1.01\%$ at $2.0\times$). In our multi-round bidding process, each server receives a modest set of candidate proposals per round (2–5 tasks), meaning the greedy efficiency ratio order closely matches the 0-1 knapsack branch choices. 2. **Both Multi-Round Bidding Strategies Beat Spill-Over Greedy**: Spill-over Greedy places high-traffic tasks first, but because it lacks cross-server regret coordination, it produces higher gaps ($7.18\% \pm 3.12\%$ at $1.0\times$, $2.39\% \pm 1.05\%$ at $2.0\times$). 3. **Random Placement Shows No Adaptation**: Random placement incurs an optimality gap of 43% **to** 47% across all capacity levels. Expanding capacity does not improve random placement (45.51% at $0.5\times \rightarrow 43.02\%$ at $2.0\times$) because random placement does not consider proximity or shortest-path routing. 4. **Smooth Reduction with Capacity**: As capacity increases from $0.5\times$ to $2.0\times$, the optimality gap to the unconstrained lower bound decreases smoothly

from 20.80% **to** 1.74%. 5. **Handling Saturated Fleets** ($0.25\times$)**:** When fleet capacity is overwhelmed (demanded resources equal 139.2% of fleet capacity), Knapsack DP places as many tasks as can fit (utilization 73.4%) and isolates the remaining 13.2 ± 1.9 **unplaced tasks**. Placed tasks experience an average access latency of 25.27 ± 3.13 ms (+32.5% above the lower bound), whereas random placement degrades to 28.13 ± 3.54 ms (+47.3%).

8.6 Workload Scaling Sensitivity

We varied workload size from 15 to 75 tasks on $|V| = 30$ with standard capacity over 10 seeds:

Table 5: Workload Size Scaling Sensitivity (Mean ± Std over 10 Seeds)

Tasks	Lower Bound Obj	Knapsack DP Obj	DP Gap (%)	Greedy Gap (%)	Spill Gap (%)	Rand Gap (%)
15	222,964 ± 30,582	223,667 ± 30,176	+0.34 ± 0.78%	+0.34 ± 0.78%	+0.51 ± 0.93%	+50.16±15.89%
25	373,363 ± 46,335	375,174 ± 45,673	+0.51 ± 0.75%	+0.53 ± 0.76%	+0.67 ± 0.83%	+49.53±11.78%
35	520,373 ± 69,139	524,407 ± 68,113	+0.82 ± 1.09%	+0.84 ± 1.15%	+1.09 ± 1.27%	+49.50±11.32%
50	738,725 ± 100,530	751,318 ± 96,209	+1.82 ± 1.76%	+1.79 ± 1.71%	+2.40 ± 2.09%	+47.64 ± 9.82%
75	1,113,646 ± 141,398	1,150,508 ± 126,983	+3.54 ± 2.92%	+3.61 ± 2.99%	+4.20 ± 2.97%	+47.37 ± 8.72%

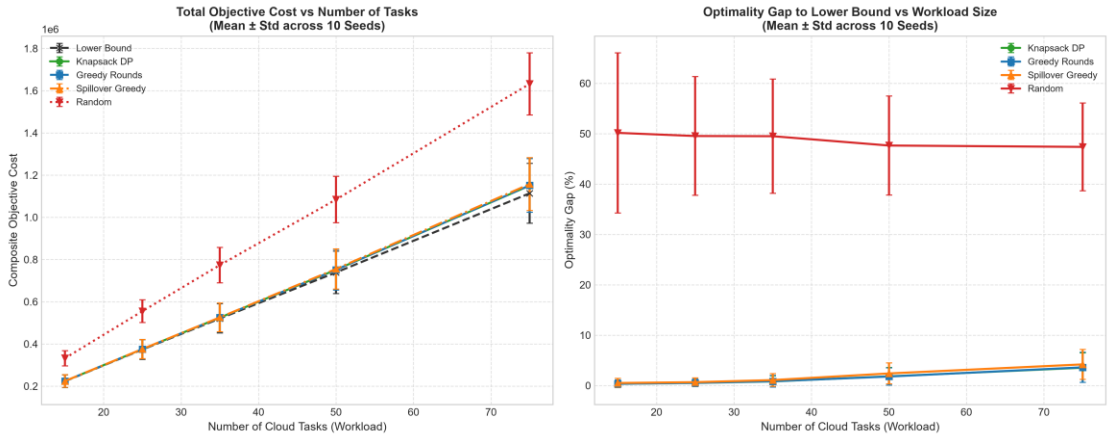


Figure 3: (Left) Total composite objective cost vs number of cloud tasks. (Right) Optimality gap relative to the unconstrained lower bound (%) with error bars.

Empirical Finding: As task count increases from 15 to 75, server contention increases, and the optimality gap to the unconstrained lower bound rises from $0.34\% \pm 0.78\%$ to $3.54\% \pm 2.92\%$. Across all workload sizes, Knapsack DP and Greedy Rounds remain closely matched and outperform Spill-over Greedy.

8.7 Lambda Sensitivity: Client Latency vs. MST Backbone Sync Trade-off

We evaluated the penalty parameter $\lambda \in [0.0, 200.0]$ across 10 random seeds on $|V| = 30$ with 35 tasks:

Table 6: Lambda Sensitivity Benchmark (Mean ± Std over 10 Seeds across Lambda in [0, 200])

Lambda	Access Lat (ms)	MST Sync Cost	Total Objective	DP Gap (%)	Random Gap (%)
0.0	18.23 ± 1.84 ms	111.2 ± 89.3	531,652 ± 52,542	+0.60 ± 0.88%	+48.19 ± 7.69%
1.0	18.24 ± 1.85 ms	98.8 ± 82.0	535,386 ± 53,207	+0.64 ± 0.91%	+49.55 ± 8.01%
5.0	18.32 ± 1.88 ms	74.8 ± 72.2	547,235 ± 56,192	+0.81 ± 1.01%	+55.68 ± 9.16%
10.0	18.49 ± 1.96 ms	57.6 ± 53.4	559,462 ± 61,089	+1.33 ± 1.32%	+64.27 ± 10.34%
25.0	18.94 ± 2.12 ms	32.2 ± 19.4	580,528 ± 70,583	+3.18 ± 2.70%	+93.84 ± 12.25%
50.0	19.14 ± 2.24 ms	26.8 ± 16.9	604,772 ± 81,146	+6.75 ± 4.90%	+146.12 ± 14.88%
100.0	19.30 ± 2.40 ms	24.8 ± 16.5	649,553 ± 102,974	+14.43 ± 9.53%	+252.50 ± 24.97%
200.0	19.45 ± 2.53 ms	23.8 ± 15.9	733,809 ± 150,274	+29.08 ± 18.45%	+465.69 ± 52.38%

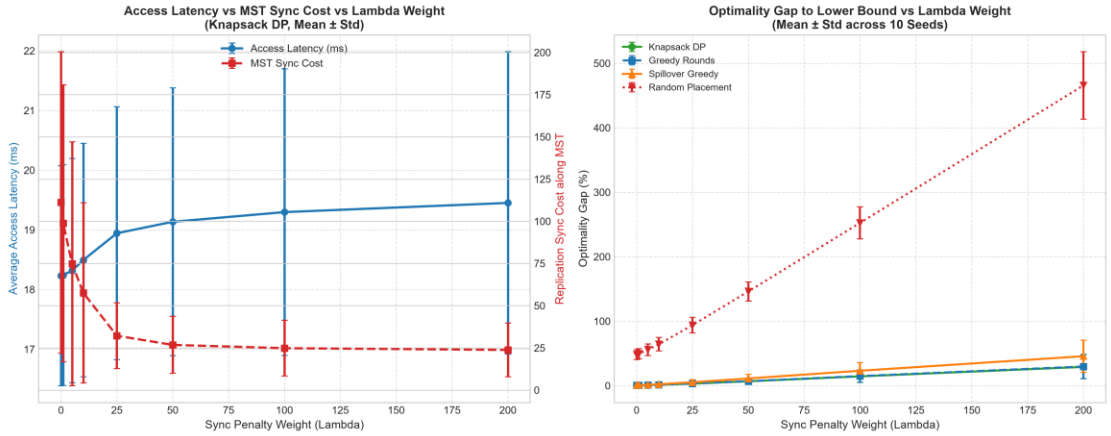


Figure 4: (Left) Access latency vs MST replication synchronization cost as penalty weight λ varies from 0 to 200. (Right) Optimality gap to unconstrained lower bound vs λ.

Empirical Finding (Key Trade-off Curve): Examining the sweep across $\lambda \in [0,200]$ reveals a clear concave trade-off curve between local client access proximity and centralized replication efficiency: - **Baseline ($\lambda = 0.0 \rightarrow 1.0$)**: At $\lambda = 1.0$, client access latency is 18.24 ± 1.85 ms (virtually identical to 18.23 ms at $\lambda = 0.0$, $+0.05\%$ difference), while MST sync cost drops from 111.2 down to 98.8 (-11.2% **reduction**). - **Moderate Weight ($\lambda = 10.0$)**: Sync cost drops steeply to 57.6 (-48.2% **reduction**), with client access latency rising modestly to 18.49 ms ($+1.4\%$ **increase**). - **High Weight ($\lambda = 25.0$)**: Sync cost reaches 32.2 (-71.0% **reduction**), with access latency rising to 18.94 ms ($+3.9\%$ **increase**). - **Diminishing Returns Plateau ($\lambda \geq 50.0$)**: For $\lambda \geq 50.0$, replication sync cost reaches diminishing returns, plateauing from 26.8 ($\lambda = 50$) to 23.8 ($\lambda = 200$, a maximum -78.6% **reduction**), while client access latency levels off at 19.45 ms ($+6.7\%$ **overall increase**). - **Lower Bound Gap Behavior**: Because λ heavily penalizes hosting servers located distant from the sync hub, server capacity bottlenecks around the hub cause the optimality gap relative to the unconstrained lower bound to rise from $+0.60\%$ at $\lambda = 0.0$ to $+29.08\%$ at $\lambda = 200.0$.

8.8 Visual Topology and Backbone Overlay

Figure 5 shows the rendered physical network layout for the 10-node small topology, showing server capacities, underlying physical mesh links, the MST communication backbone, and the traffic-weighted median **Sync Hub** (Server 0):

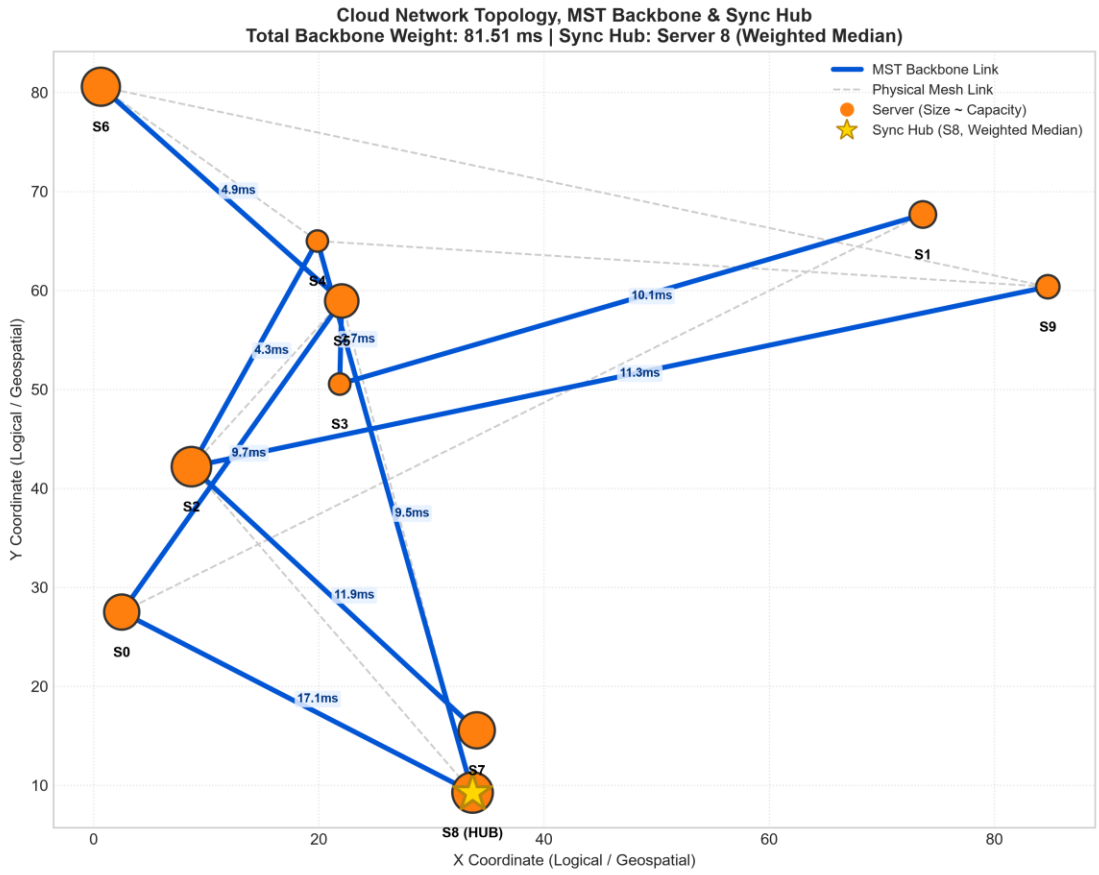


Figure 5: Visual cloud network layout showing server nodes (circle size proportional to compute capacity), underlying physical links (dashed grey), MST communication backbone links (bold blue with latency tags), and the designated traffic-weighted median Sync Hub (Server 0, marked with a star).

---

9. Limitations of the Approach

To provide an honest engineering assessment, we identify three primary limitations in our model:

1. **Heuristic Nature of Multi-Server Coordination**: Multi-server task placement under mutual capacity limits is a variant of the **Generalized Assignment Problem (GAP)** and vector bin packing, which is NP-hard in the strong sense. While our model solves each individual server's local knapsack subproblem optimally via 0-1 Knapsack DP, decomposing the global problem into iterative bidding rounds is a heuristic. It does not provide a formal polynomial-time approximation guarantee against the global joint optimum. On 60 random tiny instances (5 servers, 6 tasks) evaluated against the exact brute-force optimum, iterative knapsack DP achieved a **mean optimality gap of** 4.39% and a **median gap of** 0.00%, with a **maximum gap of** 36.75%. A share of **18 out of 60 instances (**30.0%**)** exceeded a 5% optimality gap

due to early round capacity commitments, showing that decentralized multi-round bidding can occasionally settle on locally optimal fleet configurations.

2. **Single-Resource Capacity Dimension**: Our current formulation models server capacity C_v and task demand d_j as scalar integers (e.g. CPU core or RAM slots). Production cloud datacenters require multi-dimensional vector bin packing that simultaneously constraints CPU, RAM, GPU memory, storage I/O, and egress network bandwidth. A candidate placement might satisfy CPU limits while violating memory or I/O bounds.
3. **Synthetic Network Topologies and Workloads**: Our experimental testbeds rely on synthetic graphs generated via connected random spanning trees with added link density and uniform/normal request rates. While these topologies reflect varying sizes and densities, they do not capture complex production datacenter characteristics such as hierarchical Clos/Fat-Tree fabrics, diurnal traffic spikes, or transient router queueing delays.

---

10. Algorithmic Trade-offs & Discussion

Our empirical findings clarify key practical trade-offs:

10.1 Kruskal vs. Prim for Backbone Synthesis

- **Sparse Networks ($d \leq 0.20$)**: Kruskal's algorithm is faster than Prim up to about $|V| \approx 150$ (0.110 ms vs 0.218 ms at $|V| = 50$; 0.887 ms vs 0.981 ms at $|V| = 150$) because sorting $|E| \leq 2{,}235$ edges exhibits low constant-factor overhead. However, at $|V| = 200$ sparse, Prim (2.055 ± 0.462 ms) was slightly faster than Kruskal (2.432 ± 0.722 ms), with overlapping standard deviations.
- **Dense Networks ($d \geq 0.80$)**: Prim with a min-heap is faster for $|V| \geq 100$ (0.934 ms vs 1.537 ms at $|V| = 100$; 3.012 ms vs 6.836 ms at $|V| = 200$, a $2.3\times$ speedup), as Kruskal's edge-sorting step $\mathcal{O}(E \log E)$ becomes a major bottleneck on dense graphs where $|E| \approx 16{,}000$.

10.2 Floyd-Warshall vs. Repeated Dijkstra for Routing Matrices

- **Sparse Topologies**: Repeated Dijkstra with our pure-Python `MinHeap` runs in $\mathcal{O}(V(V+E)\log V)$ and is $2.63\times$ **faster** than Floyd-Warshall at $|V| = 80$ (29.58 ms vs 77.85 ms).
- **Dense Topologies**: The ratio narrows to $1.51\times$ at $|V| = 80$ (56.74 ms vs 85.66 ms). Even on dense graphs where Floyd-Warshall is asymptotically $\mathcal{O}(V^3)$ vs Repeated Dijkstra $\mathcal{O}(V^3 \log V)$, Repeated Dijkstra remains faster in CPython. Both implementations are written from scratch in pure Python (our Dijkstra uses our custom `MinHeap`). We hypothesize that Floyd-Warshall's triple loop performing $V^3 = 512{,}000$ iterations over nested dictionary lookups (`dist[i][k] + dist[k][j]`) incurs substantial constant-factor bytecode dispatch and hash-table overhead in CPython; note that this is an unmeasured hypothesis regarding interpreter

dynamics. At small sizes ($|V| = 10$), Floyd-Warshall is faster (0.112 ms vs 0.196 ms) due to zero heap allocation overhead.

10.3 0-1 Knapsack DP vs. Greedy Rounds

- Across all experimental sweeps, Iterative Knapsack DP and Iterative Greedy Rounds are statistically indistinguishable ($6.41\% \pm 2.92\%$ vs $6.37\% \pm 2.95\%$ at $1.0\times$ capacity; $1.74\% \pm 0.99\%$ vs $1.75\% \pm 1.01\%$ at $2.0\times$). In our multi-round architecture, the 0-1 Knapsack DP is exact only per server per round on the local regret benefit, not globally across the entire fleet assignment. Because each server receives small sets of competing proposals per round (typically 2–5 tasks), sorting by greedy regret-to-demand ratio yields essentially identical assignment decisions as exact local 0-1 knapsack branches.
- Both multi-round bidding models outperform uncoordinated Spill-Over Greedy ($7.18\% \pm 3.12\%$) by using cross-server opportunity regret to coordinate which tasks claim congested servers.

---

11. Engineering Policy Recommendations

Based on our empirical results, we outline four deployment recommendations for cloud infrastructure teams:

1. **Precomputing Routing Tables with Repeated Dijkstra**: At small sizes ($|V| \leq 10$), Floyd-Warshall is faster ($0.57\times$ to $0.72\times$ the runtime of Repeated Dijkstra), and at $|V| = 25$ both approaches perform at rough parity ($\sim 1.0\times$ ratio: $1.04\times$ sparse, $0.94\times$ dense). Repeated Dijkstra with our custom `MinHeap` emerges as faster once networks reach $|V| \geq 40$ **on sparse graphs** (speedup of $1.58\times$ at $|V| = 40$, scaling to $2.63\times$ at $|V| = 80$) and $|V| \geq 60$ **on dense graphs** (speedup of $1.38\times$ at $|V| = 60$, scaling to $1.51\times$ at $|V| = 80$). We recommend Repeated Dijkstra for routing matrix precomputation on medium and large datacenter networks.
2. **Deploy an MST-Derived Replication Hub with Balanced λ Weight**: Designate the traffic-weighted median server along the MST backbone as the centralized replication sync hub. Datacenter operators can select their desired trade-off point along the empirical curve:
    - Setting $\lambda = 1.0$ cuts replication sync cost by 11.2% ($111.2 \rightarrow 98.8$) with negligible access latency degradation ($18.23 \rightarrow 18.24$ ms, $+0.05\%$).
    - Increasing to $\lambda = 10.0$ cuts sync cost by 48.2% ($111.2 \rightarrow 57.6$) with a modest $+1.4\%$ increase in client access latency (18.49 ms).
    - Weighting up to $\lambda = 25.0$ delivers a 71.0% **reduction** in replication overhead ($111.2 \rightarrow 32.2$) with a $+3.9\%$ latency trade-off (18.94 ms), beyond which sync gains plateau toward the -78.6% limit (23.8 at $\lambda = 200$).

3. **Use Regret-Based Proposal Scheduling**: Multi-round regret bidding consistently beats simple spill-over scheduling by ensuring tasks facing high displacement penalties secure appropriate hosts first.
4. **Capacity Provisioning & Fleet Sizing**: The capacity multiplier scales the raw physical compute capacities of servers across the fleet (rather than representing a fraction of total workload demand). Under standard baseline sizing ($1.0\times$ multiplier), overall fleet capacity utilization is approximately $34.4\% \pm 2.4\%$. If fleet capacities are scaled down to $0.50\times$, utilization rises to 68.8% and the latency optimality gap widens to $+20.80\%$; dropping further to $0.25\times$ causes severe saturation where an average of 13.2 tasks cannot fit. Operators should size server fleet capacity at $1.0\times$ or higher to maintain latency optimality gaps below 6.5% and avoid task rejections.

---

12. Conclusion & Future Work

In this project, our group designed, implemented, and empirically evaluated a **Cloud Task Placement Suite** from scratch in Python. By uniting Greedy Minimum Spanning Trees, Shortest Path routing algorithms, and 0-1 Knapsack Dynamic Programming: - We addressed early model limitations by introducing **iterative multi-round regret bidding**, multi-server candidate selection, and MST synchronization hub optimization. - We demonstrated that structured task placement achieves a 23.3% **to** 28.1% **client access latency reduction** relative to random placement at capacity multipliers of $0.75\times$ and above. In terms of distance from the theoretical benchmark, while random placement consistently incurs latencies 41% **to** 46% **above the unconstrained lower bound**, iterative knapsack DP stays within 1.7% **to** 6.4% of the lower bound under standard capacity provisions ($\geq 1.0\times$). - We validated our algorithms through automated unit tests against NetworkX, brute-force checks, and 10-seed empirical evaluations logging mean and standard deviation. - We delivered an interactive Jupyter Notebook dashboard that executes top-to-bottom with full inline visual outputs.

Future Work

- **Multi-Resource Vector Bin Packing**: Extending server capacities to joint vectors $\vec{C}_v = \langle \text{CPU,RAM,GPU,IOPS} \rangle$.
- **Dynamic Online Scheduling**: Incorporating online task arrival and departure models with competitive ratio bounds.
- **Resilient Routing Planes**: Integrating k-shortest paths (Yen's algorithm) to precompute instant failover paths during physical link cuts.

---

13. References

1. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

(Chapter 14: Dynamic Programming; Chapter 20: Elementary Graph Algorithms; Chapter 21: Minimum Spanning Trees; Chapter 22: Single-Source Shortest Paths; Chapter 23: All-Pairs Shortest Paths; Chapter 34: NP-Completeness).

2. **Kleinberg, J., & Tardos, É.** (2006). *Algorithm Design*. Pearson / Addison-Wesley. *(Chapter 4: Greedy Algorithms; Chapter 6: Dynamic Programming)*.
3. **Dijkstra, E. W.** (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
4. **Kruskal, J. B.** (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1), 48–50.
5. **Prim, R. C.** (1957). Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6), 1389–1401.
6. **Floyd, R. W.** (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345.
7. **Bellman, R.** (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87–90.
8. **Garey, M. R., & Johnson, D. S.** (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
9. **Meng, X., Pappas, V., & Zhang, L.** (2010). Improving the scalability of data center networks with traffic-aware virtual machine placement. In *Proceedings of IEEE INFOCOM 2010* (pp. 1–9). [verify]
10. **Karve, A., Kimbrel, T., Pacifici, G., Spreitzer, M., Steinder, M., Sviridenko, M., & Tantawi, A.** (2006). Dynamic placement for clustered web applications. In *Proceedings of the 15th International Conference on World Wide Web* (WWW '06), pp. 595–604. [verify]
11. **Shmoys, D. B., Tardos, É., & Aardal, K.** (1997). Approximation algorithms for facility location problems. In *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC '97)* (pp. 265–274). [verify]
12. **Korte, B., & Vygen, J.** (2018). *Combinatorial Optimization: Theory and Algorithms* (6th ed.). Springer. *(Chapter 22: Facility Location)*. [verify]